A3-PS026-TC073

026: AT URI Syntax Compatibility and Round-Trip Validator

Tanvi Pawar
Mushaffa Matwankar
Mentor: Prof. K.M Gajmal

AIORI Hackathon 3 · 6G & Future Networks

CONTENTS

01. Introduction

02. Literature & RFC Review

03. Proposed solution

04. Solution optimization

05. Solution architecture & design

06. Data Flow Diagram

07. Deployment Timeline

08. Bibliography

01.

Problem Description



Problem description & Background

AT URIs identify accounts and records in the AT Protocol ecosystem. A typical record URI has the form:

at://AUTHORITY/COLLECTION/RECORD_KEY

The authority is a handle or DID; the collection commonly identifies a record type, and the record key identifies a record within that collection. The AT URI specification permits DID colons in the authority, while RFC 3986 defines a generic authority structure that can include a host followed by a colon-delimited port. Standard URI parsers may therefore reject, split, or otherwise interpret a DID authority differently.



Key issues to investigate

01

Parser Behaviour

Parser behaviour is implementation-dependent; a generic parser may preserve the authority as text or split its subcomponents.

02

Syntax Validation

A parser accepting a string does not prove that the string is a valid AT URI.

03

Resolution Failure

A syntactically valid identifier may still fail resolution because the account, record, or network resource is unavailable.

02.

Literature & RFC Review



Literature & RFC Review

Independent parser implementations

The experiment should use two distinct paths: an AT-specific validator/parser and a standard URI parser (or clearly documented RFC-oriented parser configuration). Record exact library name, version, runtime, and configuration so results can be reproduced

RFC 3986 — URI Generic Syntax

Defines the generic URI components and authority grammar, including optional user information and host/port separation. It supplies the comparison baseline, but it does not define AT Protocol-specific validity rules.

AT URI Scheme Specification

Defines the AT-specific structure, allowed authority forms, character restrictions, and optional path/query/fragment behaviour. It explicitly notes that unescaped colons in DIDs conflict with standard URI parsers and that AT URIs are not fully compliant with RFC 3986 or the WHATWG URL Standard. The corpus should be pinned to a dated commit or release before testing.

03.

Proposed Solution



Proposed Solution

The prototype is a corpus-driven test harness. Each corpus row stores an identifier, expected AT-specific label, expected component values when applicable, test category, and rationale. Both parsers process the same original string; candidate transformations are then applied only to selected cases and checked for reversibility.

Core Workflow

- **Ingest**: load versioned JSON/CSV cases and validate required fields.
- **AT validation**: classify the identifier against the pinned AT URI syntax rules.
- **Generic parse:** parse the original input with an RFC-oriented URI parser; capture authority, host, port, path, query, fragment, or errors.
- **Transform:** apply candidate A and candidate B to the relevant component(s), then parse the transformed form.
- **Round trip:** decode/restore the identifier and compare original and reconstructed components byte-for-byte or character-for-character.
- **Report:** produce case-level results, aggregate metrics, and explanations for failures

04.

Solution Optimization



Solution Optimization

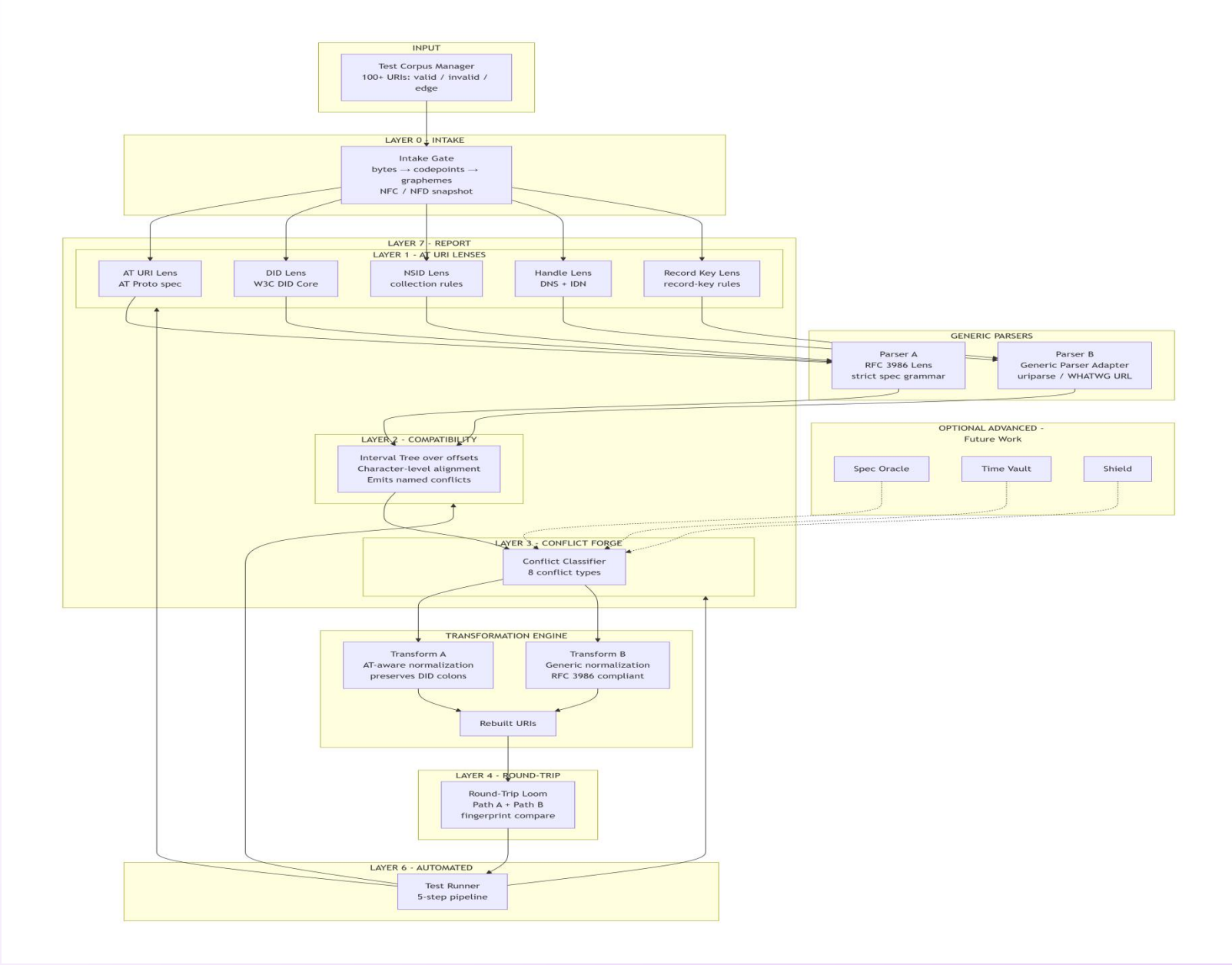
Measure	Baseline: original URI	Candidate A: authority tokenization	Candidate B: percent-encoded colon experiment
Idea	Send original at:// string directly to both parsers	Move the DID authority into a reversible opaque token in a test wrapper; preserve an exact mapping table	Test percent-encoding of DID colons in a controlled representation, then decode before AT validation
Generic parser compatibility	Send original at:// string directly to both parsers	Likely avoids authority subcomponent ambiguity in the wrapper; test empirically	May improve generic parsing, but behaviour and AT normalization constraints must be tested
Round-trip risk	No transform; baseline parse may not preserve intended split	Token collision, mapping loss, or altered wrapper context	Double decoding, normalization, or encoding-case changes
Acceptance condition	Record parser output and error	Exact original URI and components restored	Exact original URI and components restored

05.

Solution Architecture & Design



Solution Architecture & Design

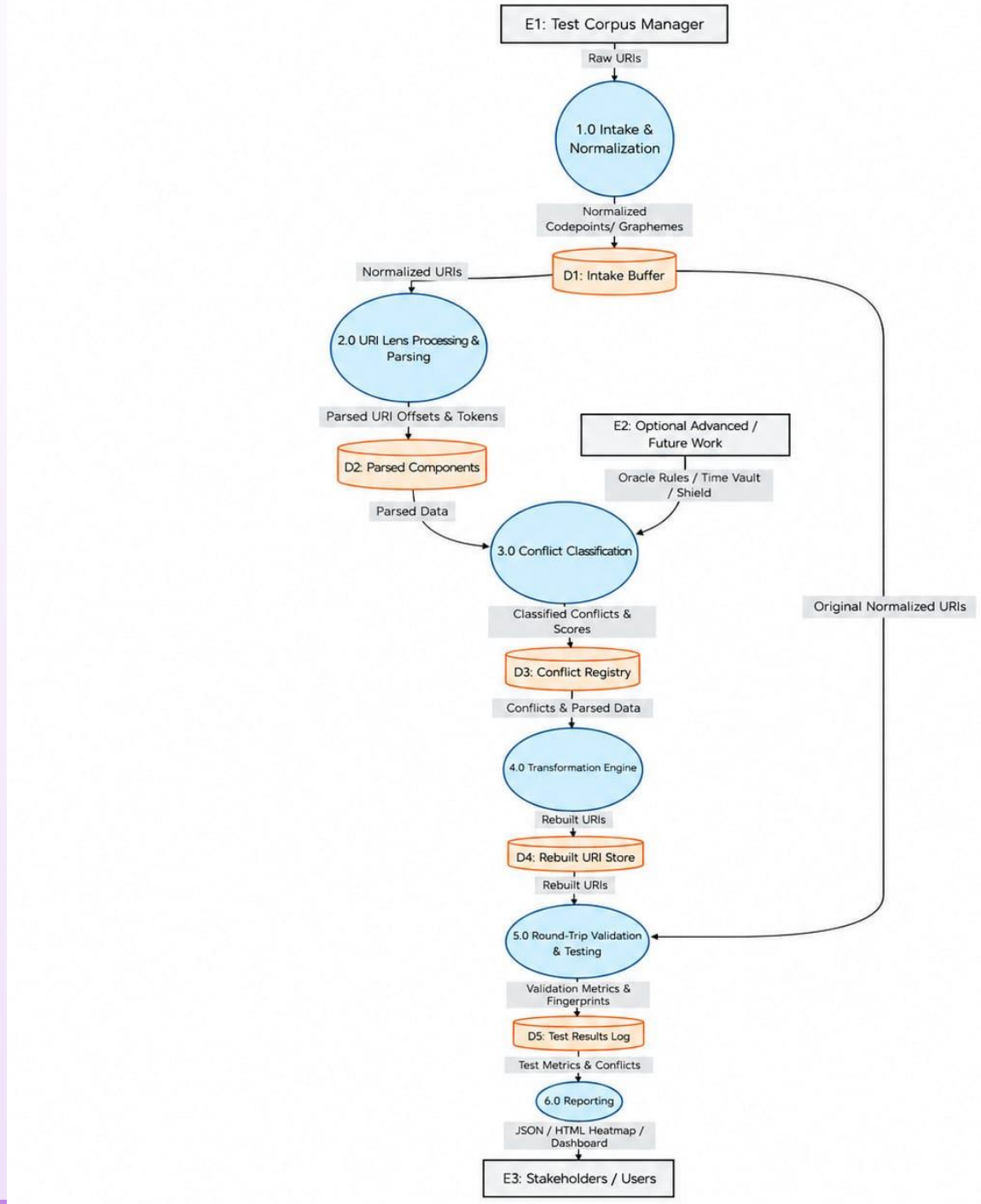


06.

Data Flow Diagram (DFD)



Data Flow Diagram (DFD)



07.

Deployment Timeline & Demonstration



Deployment Timeline & Demonstration

1. Specification Lock: Finalize AT URI specification version and RFC baseline.
2. Corpus Creation: Prepare and review 100+ test cases with expected labels.
3. Parser Implementation: Implement AT-aware and generic parsers with unit tests.
4. Transformations: Implement token wrapping and percent encoding with round-trip tests.
5. Evaluation: Test the corpus, analyze mismatches, and generate a report.
6. Demo and Review: Demonstrate pass/fail cases and limitations with a demo script and GitHub repository.

Expected Span of Completion: 18 Working Days (Approximately 3 Weeks).

08.

Bibliography



Bibliography

[1] AT Protocol. “AT URI Scheme.” Official specification. https://atproto.com/specs/at-uri-scheme .

[2] T. Berners-Lee, R. Fielding, and L. Masinter. “Uniform Resource Identifier (URI): Generic Syntax.” RFC 3986, Internet Engineering Task Force, January 2005. https://www.rfc-editor.org/rfc/rfc3986.html .

[3] Bluesky Social. AT Protocol syntax validator implementation, source repository. At urivalidation.ts .

[4] WHATWG. “URL Standard.” Available: https://url.spec.whatwg.org/.

[5] World Wide Web Consortium (W3C). “Decentralized Identifiers (DIDs) v1.0.” Available: https://www.w3.org/TR/did-core/.

[6] Python Software Foundation. “urllib.parse — Parse URLs into Components.” Available: https://docs.python.org/3/library/urllib.parse.html

Thank YOU

APP